

depending on the requirements. Some volumes are good for scaling storage volumes, some for improving performance and some for both.

Distributed GlusterFS volume: This is the default GlusterFS volume, i.e., while creating a volume, if you do not specify the type of the volume, the default option is to create a distributed type of volume. In this volume, files are randomly stored across the bricks in the volume. There is no data redundancy. The purpose for such a storage volume is to scale easily. However, this also means that a brick failure will lead to complete loss of data and one must rely on the underlying hardware for protection from data loss.

Replicated GlusterFS volume: In this volume, we overcome the data loss problem faced in the distributed volume. Here, an exact copy of the data is maintained on all the bricks. The number of replicas in the volume can be decided by the client while creating the volume. One major advantage of such a volume is that even if one brick fails, the data can still be accessed from its replica brick. Such a volume is used for better reliability and data redundancy.

Distributed replicated GlusterFS volume: This volume combines the advantages of both a distributed and a replicated volume. In this volume, files are distributed across replicated sets of bricks. The number of bricks must be a multiple of the replica count. Also, the order in which we specify the bricks matters since adjacent bricks become replicas of each other. This type of volume is used when high availability of data due to redundancy and scaling storage is required. So, if there are eight bricks and the replica count is 2, then the first two bricks become replicas of each other; then the next two and so on. This volume is denoted as 4x2. Similarly, if there are eight bricks and the replica count is 4, then four bricks become a replica of each other and we denote this volume as 2x4.

Striped GlusterFS volume: Consider a large file being stored in a brick that is frequently accessed by many clients at the same time. This will cause too much load on a single brick and might reduce its performance. In a striped volume, the data is stored in the bricks after dividing it into different stripes. So, the large file will be divided into smaller chunks (equal to the number of bricks in the volume) and each chunk is stored in a brick. Now, the load is distributed and the file can be fetched faster but no data redundancy is provided.

Reliability

To maintain availability and ensure data safety, it is recommended that you create a distributed replicated volume. We must have the replica pairs on different nodes, so that even if a node comes down, the other replica pair can be used to access data, which is immediately replicated on all pairs. Another data safety feature in GlusterFS is geo-rep. Geo-replication provides asynchronous replication

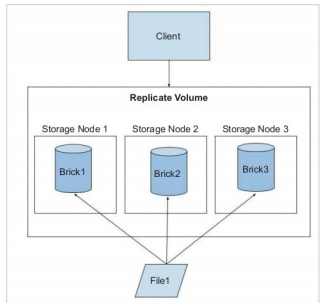


Figure 3: Replicated GlusterFS volume

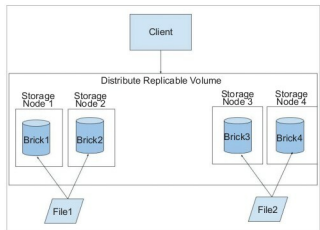


Figure 4: Distributed replicated GlusterFS volume

of data across geographically distinct locations. It mainly works across a WAN and is used to replicate the entire volume, unlike Automatic File Replication (AFR) which is intra-cluster replication (used for replicating data within the volume). This is mainly useful for backup of the entire data for disaster recovery.

GlusterFS performance

There are some translators that help in increasing the performance in GlusterFS. These, too, are present in the client *volfile*.

Readahead: Once a read request is completed, GlusterFS pre-emptively issues subsequent read requests to the server in anticipation of those requests from the user. If a sequential read request is received, the content is ready to be immediately served. If such a request is not received, the data is simply dropped.

Continued on Page 86...